UNIVERSIDAD DEL VALLE DE GUATEMALA

Ingeniera de Software 1 - Sección 30

Lynette García

Plataforma digital para la colaboración interdisciplinaria entre asociaciones estudiantiles universitarias

Angel Gabriel Sanabria Morales - 24725

Saul Esteban Castillo Arenas - 24915

Vernel Josué Hernández Cáceres - 24584

Derek Friedhelm Coronado Chilin - 24732

Samuel Antonio Robledo López - 241282

Guatemala, 2026

Resumen

El presente informe documenta el desarrollo y los resultados obtenidos durante el Sprint 2 del proyecto UVG Collab, una plataforma digital orientada a facilitar la colaboración interdisciplinaria entre asociaciones estudiantiles de la Universidad del Valle de Guatemala. El Sprint 2 se llevó a cabo del 13 al 22 de abril de 2026, comprendiendo un total de 35 puntos de historia distribuidos en ocho ítems del backlog: HU-02, HU-13, HU-07, HU-08, HU-21, HU-17, HU-25 y HT-02.

Durante este sprint se implementaron funcionalidades clave del sistema: la gestión completa del perfil académico del estudiante (incluyendo habilidades, intereses y cualidades), la creación y publicación de proyectos con roles y requisitos definidos mediante un formulario en tres pasos, la gestión de postulaciones recibidas desde la perspectiva del líder, el sistema de notificaciones en tiempo real, la búsqueda avanzada de proyectos con filtros dinámicos y el control de estado de proyectos propios. Adicionalmente, se incorporó ESLint con reglas TypeScript strict en el backend y se realizó una mejora deliberada en la consistencia visual de la interfaz.

El equipo completó el 100% de los puntos comprometidos, elevando la velocidad de 26 puntos en el Sprint 1 a 35 puntos en el Sprint 2, superando el promedio histórico de 20.67 puntos. Sin embargo, el burndown chart evidenció nuevamente un patrón de back-loading: los primeros tres días no registraron avances y la mayor parte del trabajo se concentró en el tramo final del sprint. La retrospectiva identificó como áreas de mejora la incorporación de pruebas automatizadas, la formalización del proceso de code review y la definición de entregables intermedios obligatorios para el Sprint 3.

Introducción

Contexto del proyecto

UVG Collab es una plataforma digital desarrollada como proyecto de curso en la asignatura Ingeniería de Software 1 (Sección 30) de la Universidad del Valle de Guatemala. Su propósito central es conectar a estudiantes de distintas carreras y asociaciones estudiantiles para que puedan descubrir, proponer y colaborar en proyectos interdisciplinarios dentro del ecosistema universitario. La plataforma busca resolver la falta de canales formales de visibilidad y coordinación entre iniciativas estudiantiles, permitiendo que los líderes de proyectos publiquen oportunidades con roles específicos y que los estudiantes interesados puedan postularse de manera ordenada y transparente.

El sistema está construido sobre un stack moderno: Next.js para el frontend, NestJS como framework backend, Prisma ORM con PostgreSQL como capa de persistencia, Docker para la gestión de contenedores y JWT para la autenticación. El desarrollo sigue una metodología ágil basada en sprints de una semana, con definición de historias de usuario, criterios de aceptación, estimación en puntos de historia y revisiones al cierre de cada iteración.

Alcance del Sprint 2

El Sprint 2 representa la segunda iteración de desarrollo y tiene como objetivo ampliar las funcionalidades del MVP más allá del flujo básico establecido en el Sprint 1 (autenticación, exploración de proyectos y postulación). En esta iteración se abordan las siguientes áreas funcionales:

- Gestión del perfil académico del estudiante, incluyendo la edición de información personal, carrera, habilidades, intereses y cualidades (HU-02, HU-13).
- Creación y publicación de proyectos con roles, cupos y requisitos de habilidad definidos por el líder (HU-07).
- Gestión de postulaciones recibidas por el líder del proyecto, con capacidad de aceptar o rechazar con comentario opcional (HU-08).
- Sistema de notificaciones que comunica al postulante el resultado de su solicitud en tiempo real dentro de la plataforma (HU-21).
- Búsqueda avanzada de proyectos mediante filtros combinados por tipo, modalidad, organización y habilidad requerida (HU-17).
- Vista de gestión de proyectos propios con control de cambio de estado siguiendo transiciones válidas definidas en el sistema (HU-25).
- Configuración de ESLint con reglas TypeScript strict en el backend, mejoras de consistencia visual y pruebas manuales del flujo completo (HT-02).

Product Backlog

ID	Historia de Usuario	Descripción	Actor principal	Prioridad	Estado	Sprint	Notas (estado del código)
Historias Técnicas (HT)							

HT-01	Preparación presentación, guion y revisión (S1)	Elaborar presentación, guion, flujo y revisar errores para la entrega del Sprint 1.	Equipo	Alta	✓ **Completada**	Sprint 1	
HT-02	Tareas transversales Sprint 2	ESLint backend, mejora consistencia visual UI, pruebas manuales flujo completo y preparación entrega Sprint 2.	Equipo	Alta	✓ **Completada**	Sprint 2	*eslint.config.mjs ya existía en frontend desde Sprint 1.*
HT-03	Tareas transversales Sprint 3	Tareas transversales que surjan durante el Sprint 3.	Equipo	Alta	**— No iniciada**	—	
HT-04	Tareas transversales Sprint 4	Tareas transversales que surjan durante el Sprint 4.	Equipo	Alta	**— No iniciada**	—	
Infraestructura y Configuración							
HU-00	Configuración e infraestructura del proyecto	Configurar entorno de trabajo: monorepo, BD PostgreSQL, Docker, Prisma ORM, seed inicial con 18 enums y 30+ modelos.	Equipo	Alta	✓ **Completada**	Sprint 1	*Monorepo, Docker, Prisma, seed.ts completamente funcional.*
Autenticación y Perfil de Usuario							
HU-01	Registro e inicio de sesión	Como estudiante deseo registrarme e iniciar sesión para acceder a la plataforma.	Estudiante	Alta	✓ **Completada**	Sprint 1	*POST /auth/login, POST /auth/register, JWT, validación @uvg.edu.gt.*
HU-02	Completar perfil académico	Como estudiante deseo completar mi perfil con carrera,	Estudiante	Alta	✓ **Completada**	Sprint 2	*Endpoints GET/PATCH /usuarios/me/perfil y PUT habilidades/intereses/cualidades. Form completo con React Hook Form + Zod.*

		habilidades, intereses y cualidades.					
HU -13	Editar perfil académico	Como estudiante deseo editar mi perfil para mantener actualizadas mis habilidades e intereses.	Estudiante	Alta	✓ **Completada**	Sprint 2	*Reutiliza formulario HU-02 en modo edición con datos precargados desde bootstrap endpoint.*
HU -14	Recuperación de contraseña	Como usuario deseo recuperar mi contraseña mediante correo electrónico.	Estudiante	Media	**— No iniciada**	—	*UI en /recuperar-contrasena existe. Backend sin endpoint de recuperación.*
HU -15	Subir y actualizar foto de perfil	Como estudiante deseo subir una foto de perfil para personalizar mi cuenta.	Estudiante	Baja	**— No iniciada**	—	*updateFotoUrl() existe en users.service. Cloudinary configurado. Falta UI de subida.*
HU -16	Cerrar sesión de forma segura	Como usuario deseo cerrar sesión para proteger mi cuenta en dispositivos compartidos.	Estudiante	Alta	✓ **Completada**	Sprint 1	*Botón LogOut en DashboardLayout.tsx elimina token y redirige a /login.*
HU -61	Autocompletar correo institucional con carné ★	Como estudiante deseo que el sistema genere mi correo a partir de apellido y carné.	Estudiante	Media	**— No iniciada**	—	*Nueva HU derivada de feedback de usuarios en presentación del Sprint 1.*
Exploración y Descubrimiento de Proyectos							
HU -03	Explorar proyectos publicados	Como estudiante deseo explorar proyectos disponibles para encontrar oportunidades de colaboración.	Estudiante	Alta	✓ **Completada**	Sprint 1	*GET /proyectos con filtro por título. Vista /dashboard/proyectos con grid y tarjetas.*

HU-04	Ver detalle de proyecto	Como estudiante deseo ver la información completa de un proyecto antes de postularme.	Estudiante	Alta	✓ **Completada**	Sprint 1	*GET /proyectos/:id. Vista detalle con roles, habilidades, hitos (hitos-section.tsx).*
HU-17	Búsqueda avanzada de proyectos	Como estudiante deseo buscar proyectos por filtros combinados: tipo, modalidad, organización y habilidades.	Estudiante	Alta	✓ **Completada**	Sprint 2	*GET /proyectos extendido con query params adicionales. Panel de filtros colapsable en UI.*
HU-18	Proyectos recomendados según perfil	Como estudiante deseo recibir sugerencias de proyectos compatibles con mis habilidades e intereses.	Estudiante	Media	**— No iniciada**	—	
HU-19	Filtrar proyectos por organización	Como estudiante deseo filtrar proyectos por la organización que los publica.	Estudiante	Media	**— No iniciada**	—	*Parcialmente cubierto por HU-17 (filtro organizacionId).*
HU-59	Proyectos destacados ★	Como estudiante deseo ver proyectos destacados o populares en la plataforma.	Estudiante	Media	**— No iniciada**	—	*Nueva HU derivada de feedback de usuarios en presentación del Sprint 1.*
Postulaciones							
HU-05	Postularse a un rol en proyecto	Como estudiante deseo postularme a un rol específico dentro de un proyecto.	Estudiante	Alta	✓ **Completada**	Sprint 1	*POST /postulaciones. Vista /postular/[rolId] con form Zod, validaciones frontend y backend.*
HU-06	Consultar mis postulaciones	Como estudiante deseo ver el historial y	Estudiante	Alta	✓ **Completada**	Sprint 1	*GET /postulaciones/mis-postulaciones. Vista con badges de estado por postulación.*

		estado de todas mis postulaciones.					
HU-20	Cancelar postulación enviada	Como estudiante deseo cancelar una postulación que aún está en estado pendiente.	Estudiante	Media	**— No iniciada**	—	
HU-21	Notificación de resultado de postulación	Como estudiante deseo ser notificado dentro de la plataforma cuando mi postulación sea aceptada o rechazada.	Estudiante	Media	**✓ Completada**	Sprint 2	*Módulo NotificationsService completo. Endpoints GET/PATCH notificaciones. Campana en navbar.*
Creación y Gestión de Proyectos (Líder)							
HU-07	Crear y publicar proyecto	Como líder de asociación deseo crear y publicar proyectos con roles y requisitos de habilidad definidos.	Líder	Alta	**✓ Completada**	Sprint 2	*POST /proyectos extendido con CreateProjectFullDto. Formulario multi-paso 3 pasos. Transacción Prisma.*
HU-22	Editar proyecto existente	Como líder deseo editar la información de un proyecto ya publicado.	Líder	Media	**— No iniciada**	—	
HU-23	Definir roles y requisitos del proyecto	Como líder deseo definir los roles disponibles junto con habilidades requeridas.	Líder	Media	**— No iniciada**	—	*Modelos RolProyecto y RequisitoHabilidad en schema. Cubierto parcialmente por HU-07.*
HU-24	Definir hitos y cronograma del proyecto	Como líder deseo establecer hitos con fechas estimadas de entrega dentro del proyecto.	Líder	Media	**— No iniciada**	—	*Modelo Hito existe en schema. hitos-section.tsx solo lectura en frontend.*

HU -25	Cambiar estado del proyecto	Como líder deseo cambiar el estado de mis proyectos siguiendo transiciones válidas (borrador → publicado…).	Líder	Media	**✓ Completada**	Sprint 2	*PATCH /proyectos/:id/estado con tabla de transiciones. Vista Mis Proyectos con selector.*
HU -26	Archivar o eliminar proyecto	Como líder deseo archivar o eliminar un proyecto inactivo para mantener limpia la plataforma.	Líder	Baja	**— No iniciada**	—	*Campo eliminadoEn en modelo (soft delete). Sin endpoint ni UI.*
Gestión de Postulaciones (Líder)							
HU -08	Gestionar postulaciones recibidas	Como líder deseo aceptar o rechazar postulaciones para formar el equipo de trabajo de mi proyecto.	Líder	Alta	**✓ Completada**	Sprint 2	*GET /proyectos/:id/postulaciones. PATCH /postulaciones/:id/estado. Vista agrupada por rol.*
HU -27	Ver perfil del postulante	Como líder deseo ver el perfil completo de un postulante para evaluar su idoneidad.	Líder	Media	**— No iniciada**	—	
HU -28	Enviar mensaje al postulante	Como líder deseo enviar un mensaje directo al postulante durante el proceso de evaluación.	Líder	Baja	**— No iniciada**	—	
Gestión de Tareas y Seguimiento							
HU -09	Gestión de tareas del proyecto	Como colaborador deseo gestionar las tareas que se me han asignado para organizar mi trabajo.	Colaborador	Media	**— No iniciada**	—	*Módulo tasks existe como stub. Modelo Tarea definido en schema.*

HU -29	Crear y asignar tareas a colaboradores	Como líder deseo crear tareas y asignarlas a los miembros del equipo.	Líder	Media	**— No iniciada**	—	
HU -30	Visualizar tablero Kanban de tareas	Como colaborador deseo ver las tareas en un tablero Kanban para tener visibilidad del avance.	Colaborador	Media	**— No iniciada**	—	
HU -31	Comentar en tareas	Como colaborador deseo agregar comentarios en las tareas para comunicar avances.	Colaborador	Baja	**— No iniciada**	—	
HU -32	Seguimiento de avance general	Como líder deseo visualizar el porcentaje de avance del proyecto según tareas completadas.	Líder	Media	**— No iniciada**	—	
Evidencias y Validación							
HU -10	Subir evidencias de trabajo	Como colaborador deseo subir evidencias para documentar mi participación en el proyecto.	Colaborador	Baja	**— No iniciada**	—	*Módulo evidence existe como stub.*
HU -11	Validar horas de participación	Como coordinador deseo validar las horas reportadas por los estudiantes para beca y extensión.	Coordinador	Baja	**— No iniciada**	—	
HU -33	Revisar evidencias subidas	Como líder deseo revisar y aprobar o rechazar las evidencias enviadas por	Líder	Baja	**— No iniciada**	—	

		los colaboradores.					
HU -34	Generar certificado de participación	Como estudiante deseo generar un certificado digital que avale mi participación en un proyecto.	Estudiante	Baja	**— No iniciada**	—	
HU -35	Exportar historial de participación	Como estudiante deseo exportar un resumen en PDF de mis proyectos y horas de participación.	Estudiante	Baja	**— No iniciada**	—	
Organizaciones Estudiantiles							
HU -36	Crear organización estudiantil	Como líder deseo registrar una organización estudiantil en la plataforma.	Líder	Media	**— No iniciada**	—	*Modelo Organizacion existe en schema con relaciones definidas.*
HU -37	Gestionar miembros de organización	Como líder deseo agregar o remover miembros de la organización.	Líder	Media	**— No iniciada**	—	
HU -38	Ver perfil público de organización	Como estudiante deseo ver el perfil de una organización con sus proyectos activos.	Estudiante	Media	**— No iniciada**	—	
Comunicación y Notificaciones							
HU -12	Notificaciones del sistema	Como usuario deseo recibir notificaciones sobre todos los eventos relevantes de la plataforma.	Estudiante	Media	**— No iniciada**	—	*Módulo notifications como stub en Sprint 1; implementado completamente en Sprint 2 (HU-21).*
HU -39	Configurar preferencias de notificación	Como usuario deseo configurar qué tipo de	Estudiante	Baja	**— No iniciada**	—	

		notificaciones deseo recibir.					
HU -40	Mensajería interna entre usuarios	Como usuario deseo enviar y recibir mensajes privados dentro de la plataforma.	Estudiante	Baja	**— No iniciada**	—	
HU -41	Anuncios dentro de un proyecto	Como líder deseo publicar anuncios visibles para todos los colaboradores del proyecto.	Líder	Baja	**— No iniciada**	—	
Panel de Administración							
HU -42	Panel de administración general	Como administrador deseo un panel para gestionar usuarios, proyectos y organizaciones.	Administrador	Media	**— No iniciada**	—	
HU -43	Gestión de catálogos del sistema	Como administrador deseo gestionar los catálogos de carreras, habilidades e intereses.	Administrador	Media	**— No iniciada**	—	*Módulo catalogs con endpoints GET carreras, habilidades, intereses ya operativos.*
HU -44	Moderación de contenido	Como administrador deseo revisar y moderar proyectos u organizaciones reportados.	Administrador	Baja	**— No iniciada**	—	
HU -45	Gestión de roles de usuario	Como administrador deseo asignar o revocar roles (estudiante, líder, coordinador, admin).	Administrador	Media	**— No iniciada**	—	*Enum RolUsuario definido en schema.*
Reportes y Analítica							

HU -46	Dashboard de métricas para líderes	Como líder deseo ver métricas de mi proyecto: postulaciones recibidas, roles cubiertos, avance.	Líder	Baja	**— No iniciada**	—	
HU -47	Reporte de horas por estudiante	Como coordinador deseo generar un reporte de horas beca y extensión por estudiante.	Coordinador	Baja	**— No iniciada**	—	
HU -48	Estadísticas generales de la plataforma	Como administrador deseo visualizar estadísticas globales de uso de la plataforma.	Administrador	Baja	**— No iniciada**	—	
HU -49	Exportar reportes a PDF	Como líder o administrador deseo exportar reportes en formato PDF.	Líder	Baja	**— No iniciada**	—	
Portafolio y Reconocimiento							
HU -50	Portafolio público del estudiante	Como estudiante deseo un portafolio público con mis proyectos completados y certificados.	Estudiante	Baja	**— No iniciada**	—	
HU -51	Historial de proyectos completados	Como estudiante deseo consultar el historial completo de mis proyectos.	Estudiante	Baja	**— No iniciada**	—	
HU -52	Valoraciones entre colaboradores	Como colaborador deseo calificar mi experiencia en un proyecto finalizado.	Colaborador	Baja	**— No iniciada**	—	
HU -60	Feedback de actividades	Como colaborador	Colaborador	Baja	**— No iniciada**	—	*Nueva HU derivada de feedback de usuarios en presentación del Sprint 1.*

	del proyecto ★	deseo dar feedback sobre las actividades del proyecto en curso.					
Social y Descubrimiento							
HU -58	Seguir a creadores de proyectos ★	Como estudiante deseo seguir a líderes para recibir actualizaciones sobre sus nuevas publicaciones.	Estudiante	Baja	**— No iniciada**	—	*Nueva HU derivada de feedback de usuarios en presentación del Sprint 1.*
Calendario y Planificación							
HU -53	Calendario de actividades del proyecto	Como colaborador deseo ver un calendario con los hitos y fechas límite del proyecto.	Colaborador	Baja	**— No iniciada**	—	
HU -54	Recordatorios de fechas límite	Como colaborador deseo recibir recordatorios automáticos antes del vencimiento de tareas.	Colaborador	Baja	**— No iniciada**	—	
Configuración y Experiencia de Usuario							
HU -55	Tema visual (modo claro/oscuro)	Como usuario deseo alternar entre modo claro y oscuro según mi preferencia.	Estudiante	Media	**— No iniciada**	—	
HU -56	Diseño responsivo para dispositivos móviles	Como usuario deseo acceder a la plataforma desde celular o tablet con experiencia optimizada.	Estudiante	Media	**— No iniciada**	—	
HU -57	Tutorial / guía de uso interactiva ★	Como usuario nuevo deseo acceder a un tutorial o guía interactiva	Estudiante	Media	**— No iniciada**	—	*Nueva HU derivada de feedback de usuarios en presentación del Sprint 1.*

		dentro del sistema.					

Sprint Backlog

Tipo	ID	Título	SP	Hrs Est.	Responsable	Inicio	Fin	Prioridad	Estado
Historia	**HU-02**	**Completar perfil académico**	**5**		**Angel Sanabria**	**13/04**	**15/04**	**Alta**	**Done**
Subtarea	T-27	Revisión/ajuste endpoints perfil	1	3	Angel Sanabria	13/04	13/04	Alta	**Done**
Subtarea	T-28	Formulario perfil /dashboard/perfil	2	5	Samuel Robledo	13/04	14/04	Alta	**Done**
Subtarea	T-29	Integración perfil con backend	2	3	Samuel Robledo	15/04	15/04	Alta	**Done**
Historia	**HU-13**	**Editar perfil académico**	**2**		**Samuel Robledo**	**15/04**	**16/04**	**Alta**	**Done**
Subtarea	T-30	Modo edición del perfil precargado	2	3	Samuel Robledo	15/04	16/04	Alta	**Done**
Historia	**HU-07**	**Crear y publicar proyecto**	**8**		**Vernel Hernández**	**13/04**	**17/04**	**Alta**	**Done**
Subtarea	T-31	POST /proyectos con roles y habilidades	3	5	Angel Sanabria	13/04	14/04	Alta	**Done**

Subtarea	T-32	PATCH /proyectos/:id/estado	1	2	Angel Sanabria	15/04	15/04	Media	**Done**
Subtarea	T-33	Formulario creación proyecto (3 pasos)	3	6	Vernel Hernández	13/04	15/04	Alta	**Done**
Subtarea	T-34	Integración y validaciones del formulario	1	3	Vernel Hernández	16/04	16/04	Alta	**Done**
Historia	**HU-08**	**Gestionar postulaciones recibidas**	**5**		**Saul Castillo**	**14/04**	**17/04**	**Alta**	**Done**
Subtarea	T-35	GET /proyectos/:id/postulaciones	2	3	Saul Castillo	14/04	14/04	Alta	**Done**
Subtarea	T-36	Verificación PATCH aceptar/rechazar	1	2	Saul Castillo	15/04	15/04	Alta	**Done**
Subtarea	T-37	Vista gestión de postulaciones (líder)	2	5	Saul Castillo	15/04	17/04	Alta	**Done**
Historia	**HU-21**	**Notificación resultado de postulación**	**3**		**Derek Coronado**	**14/04**	**16/04**	**Media**	**Done**
Subtarea	T-38	Módulo de notificaciones real	1	3	Derek Coronado	14/04	14/04	Media	**Done**
Subtarea	T-39	Notificación automática al gestionar postulación	1	3	Derek Coronado	15/04	15/04	Media	**Done**
Subtarea	T-40	Campana de notificaciones (navbar)	1	4	Derek Coronado	15/04	16/04	Media	**Done**
Historia	**HU-17**	**Búsqueda avanzada de proyectos**	**5**		**Saul Castillo**	**16/04**	**18/04**	**Alta**	**Done**
Subtarea	T-41	GET /proyectos con filtros adicionales	2	4	Saul Castillo	17/04	18/04	Alta	**Done**
Subtarea	T-42	Panel de filtros en vista proyectos	3	5	Derek Coronado	16/04	18/04	Alta	**Done**
Historia	**HU-25**	**Cambiar estado del proyecto (Mis Proyectos)**	**3**		**Vernel Hernández**	**16/04**	**18/04**	**Media**	**Done**
Subtarea	T-43	GET /proyectos/mis-proyectos	1	3	Vernel Hernández	16/04	17/04	Media	**Done**
Subtarea	T-44	Vista Mis Proyectos + control estado	2	4	Vernel Hernández	17/04	18/04	Media	**Done**
Historia	**HT-02**	**Tareas transversales Sprint 2**	**5**		**Equipo**	**13/04**	**19/04**	**Alta**	**Done**
Subtarea	T-45	Configuración ESLint backend	1	3	Angel Sanabria	15/04	15/04	Alta	**Done**
Subtarea	T-46	Consistencia visual UI	1	3	Samuel Robledo	17/04	17/04	Media	**Done**
Subtarea	T-47	Pruebas flujo completo Sprint 2	1	4	Vernel Hernández	18/04	19/04	Alta	**Done**
Subtarea	T-48	Corrección de errores generales	1	3	Derek Coronado	18/04	19/04	Media	**Done**

Subtarea	T-49	Preparación presentación y guion	1	3	Saul Castillo	18/04	19/04	Media	**Done**

Incremento

Vínculo al repositorio

Repositorio: https://github.com/asanabria-2021067/proyecto-ingenieria-software

El código desarrollado y funcional del Sprint 2 se encuentra en la rama principal (origin/main) del repositorio indicado. Los commits correspondientes a este sprint parten desde el 6 de abril de 2026 e integran el flujo completo de las historias de usuario HU-02, HU-13, HU-07, HU-08, HU-21, HU-17, HU-25 y la historia técnica HT-02.

Aplicación hosteada: http://158.23.57.118/

Lista de funcionalidades planificadas que se terminaron completamente

1. HU-02: Completar perfil académico

T-27 — Revisión y ajuste de endpoints de perfil existentes

Se verificaron y ajustaron los endpoints del módulo de usuarios para que cubran la totalidad del perfil académico. El endpoint GET /usuarios/me/perfil/bootstrap es el punto de entrada del formulario de edición: ejecuta en paralelo la consulta del perfil actual del usuario y cuatro catálogos (carreras, habilidades, intereses y cualidades) devolviendo todo en una sola respuesta, lo que evita múltiples peticiones en cascada desde el frontend.

```
async getProfileBootstrap(userId: number) {
  const user = await this.prisma.usuario.findUnique({
    where: { idUsuario: userId },
    include: { perfil: { include: { carrera: true } },
               habilidades: { include: { habilidad: true } },
               intereses: { include: { interes: true } },
               cualidades: { include: { cualidad: true } } },
  });
  const [carreras, habilidadesCatalog, interesesCatalog, cualidadesCatalog] =
    await Promise.all([
      this.prisma.carrera.findMany({ where: { estado: 'ACTIVA' } }),
      this.prisma.habilidad.findMany(),
```

```
      this.prisma.interes.findMany(),
      this.prisma.cualidad.findMany(),
    ]);
  return { profile: { ...mappedProfile }, catalogs: { carreras, habilidadesCatalog, ...
} };
}
```

El endpoint PATCH /usuarios/me/perfil ejecuta una transacción que actualiza simultáneamente la tabla usuario (nombre, apellido, foto) y la tabla perfilEstudiante (carrera, semestre, biografía, enlaces, disponibilidad, horas beca/extensión requeridas). Las colecciones de habilidades, intereses y cualidades se actualizan a través de endpoints PUT dedicados que reemplazan por completo la colección existente mediante delete-then-create dentro de una transacción Prisma.

```
async replaceHabilidades(userId, habilidades) {
  return this.prisma.$transaction(async (tx) => {
    await tx.usuarioHabilidad.deleteMany({ where: { idUsuario: userId } });
    if (habilidades.length > 0) {
      await tx.usuarioHabilidad.createMany({
        data: habilidades.map(h => ({
          idUsuario: userId, idHabilidad: h.idHabilidad,
          nivelHabilidad: h.nivelHabilidad,
          aniosExperiencia: h.aniosExperiencia,
        })),
      });
    }
    return { count: habilidades.length };
  });
}
```

T-28 / T-29 — Formulario completo de perfil e integración con backend

Se desarrolló el formulario de perfil académico accesible desde el panel del usuario. El componente CompleteProfileDialog consume el endpoint bootstrap para precargar todos los selectores (carrera, habilidades, intereses, cualidades) con los datos actuales del usuario. El formulario integra React Hook Form con Zod para validación en tiempo real y TanStack Query para el envío de mutaciones a cada endpoint correspondiente. Los estados de carga, éxito y error se comunican al usuario mediante mensajes en línea.

2. HU-13: Editar perfil académico

T-30 — Modo edición del perfil con datos precargados

El formulario de perfil implementado en HU-02 soporta el modo de edición reutilizando los mismos componentes y endpoints. Al abrir el formulario en modo edición, el hook useQuery consulta GET

/usuarios/me/perfil/bootstrap y precarga todos los campos con los valores vigentes del usuario. Una vez guardados los cambios, la mutación invalida el caché de TanStack Query correspondiente al perfil para que el dashboard refleje los datos actualizados.

```
const { data: bootstrap } = useQuery({
  queryKey: ['perfil-bootstrap'],
  queryFn: () => apiFetch('/usuarios/me/perfil/bootstrap'),
});

// Precarga de datos al montar el formulario
useEffect(() => {
  if (!bootstrap) return;
  const { profile } = bootstrap;
  setValue('nombreCompleto', profile.nombreCompleto);
  setValue('idCarrera', profile.idCarrera);
  setValue('biografia', profile.biografia);
  setValue('habilidades', profile.habilidades);
  // ... resto de campos
}, [bootstrap]);
```

3. HU-07: Crear y publicar proyecto

T-31 — Extensión del endpoint POST /proyectos con roles y habilidades

Se diseñó el DTO CreateProjectFullDto que permite crear en una sola solicitud el proyecto, sus roles, los requisitos de habilidad por rol y las organizaciones vinculadas. Toda la operación se ejecuta dentro de una transacción Prisma para garantizar la consistencia de los datos. El campo accion determina el estado inicial del proyecto: BORRADOR si el líder desea seguir editando, o EN_REVISION si desea enviarlo directamente a aprobación.

```
async createFull(data: CreateProjectFullDto, creadoPor: number) {
  const { organizacionesIds, roles, accion, ...rest } = data;
  const estadoProyecto = accion === 'EN_REVISION'
    ? EstadoProyecto.EN_REVISION : EstadoProyecto.BORRADOR;
  return this.prisma.$transaction(async (tx) => {
    const proyecto = await tx.proyecto.create({
      data: {
        ...rest, estadoProyecto, creadoPor,
        ...(organizacionesIds?.length && {
          organizaciones: { create: organizacionesIds.map(id => ({ idOrganizacion: id
})) },
        }),
        ...(roles?.length && {
```

```
          roles: { create: roles.map(rol => ({
            nombreRol: rol.nombreRol, cupos: rol.cupos,
            ...(rol.requisitos?.length && {
              requisitos: { create: rol.requisitos.map(req => ({
                idHabilidad: req.idHabilidad, nivelMinimo: req.nivelMinimo,
                obligatorio: req.obligatorio,
              })) },
            }),
          })) },
        }),
      },
    });
    if (accion === 'EN_REVISION') {
      await this._crearRevisionPendiente(tx, proyecto.idProyecto, 1);
      await this.notifications.notifyAdmins({ tipoNotificacion: 'PROYECTO_EN_REVISION',
... }, tx);
    }
    return proyecto;
  });
}
```

T-32 — Implementación del endpoint PATCH /proyectos/:id/estado

Se implementó el endpoint protegido PATCH /proyectos/:id/estado que permite al creador del proyecto cambiar el estado siguiendo transiciones válidas definidas en una tabla de transiciones permitidas. El servicio valida que el usuario sea el creador mediante _requireOwner y que la transición solicitada esté permitida para el estado actual. Cuando el nuevo estado es PUBLICADO se registra automáticamente la fechaPublicacion.

```
async changeEstado(id: number, userId: number, nuevoEstado: EstadoProyectoCreador) {
  const proyecto = await this._requireOwner(id, userId);
  const permitidos = TRANSICIONES_PERMITIDAS[proyecto.estadoProyecto] ?? [];
  if (!permitidos.includes(nuevoEstado as EstadoProyecto)) {
    throw new BadRequestException(
      `Transición no permitida: ${proyecto.estadoProyecto} -> ${nuevoEstado}`
    );
  }
  return this.prisma.proyecto.update({
    where: { idProyecto: id },
    data: {
      estadoProyecto: nuevoEstado as EstadoProyecto,
      fechaActualizacion: new Date(),
      ...(nuevoEstado === 'PUBLICADO' && { fechaPublicacion: new Date() }),
    },
  });
}
```

T-33 / T-34 — Formulario de creación de proyecto en tres pasos e integración

Se desarrolló el formulario de creación y edición de proyectos en la ruta /dashboard/projects/mine/form, estructurado en tres pasos con animación de transición: Step1 captura la información general del proyecto (título, descripción, tipo, modalidad, objetivos, contexto académico, URL de recurso externo y fechas estimadas), Step2 permite definir los roles disponibles del proyecto junto con sus cupos, descripción, carrera requerida y requisitos de habilidad con nivel mínimo y obligatoriedad, y Step3 presenta un resumen de lo ingresado y expone dos acciones: guardar como borrador o enviar directamente a revisión. El formulario soporta también el modo de edición cuando se pasa el parámetro id en la URL, en cuyo caso precarga los datos del proyecto existente mediante GET /proyectos/:id/owner.

```
const submit = async (accion: 'BORRADOR' | 'EN_REVISION') => {
  const clientErrors = validateBeforeSubmit();
  if (clientErrors.length > 0) {
    uvgSwal.fire({ icon: 'warning', title: 'Campos incompletos', ... });
    return;
  }
  if (editId) {
    await updateProject(editId, buildDataPayload());
    if (accion === 'EN_REVISION') await submitProjectForReview(editId);
  } else {
    await createProject({ ...buildDataPayload(), accion });
  }
  await queryClient.invalidateQueries({ queryKey: ['mis-proyectos'] });
  router.push('/dashboard/projects/mine');
};
```

4. HU-08: Gestionar postulaciones recibidas

T-35 — Endpoint GET /proyectos/:id/postulaciones

Se implementó el endpoint protegido GET /proyectos/:id/postulaciones que devuelve todas las postulaciones recibidas para un proyecto específico. El servicio invoca _requireOwner para verificar que el usuario autenticado sea el creador del proyecto antes de exponer los datos. La respuesta incluye el nombre, apellido y correo del postulante, el nombre del rol al que se postuló, la justificación enviada, el estado actual de la postulación y las fechas de postulación y resolución.

```
async findPostulacionesByProject(idProyecto: number, userId: number) {
  await this._requireOwner(idProyecto, userId);
  return this.prisma.postulacion.findMany({
    where: { rolProyecto: { idProyecto } },
    include: {
```

```
      postulante: {
        select: { idUsuario: true, nombre: true, apellido: true, correo: true },
      },
      rolProyecto: {
        select: { idRolProyecto: true, nombreRol: true },
      },
    },
    orderBy: { fechaPostulacion: 'desc' },
  });
}
```

T-36 — Verificación y ajuste del endpoint PATCH /postulaciones/:id/estado

Se revisó y reforzó el endpoint PATCH /postulaciones/:id/estado para garantizar dos validaciones críticas: que la postulación se encuentre en estado PENDIENTE (se lanza BadRequestException si ya fue resuelta) y que el usuario autenticado sea el creador del proyecto asociado al rol de la postulación (se lanza ForbiddenException en caso contrario). Al resolver la postulación se registran la fecha de resolución, el usuario resolutor y el comentario opcional ingresado por el líder.

```
async updateEstado(id: number, dto: UpdateEstadoPostulacionDto, resolutorId: number) {
  if (postulacion.estadoPostulacion !== 'PENDIENTE')
    throw new BadRequestException('Esta postulación ya fue resuelta');
  if (postulacion.rolProyecto.proyecto.creadoPor !== resolutorId)
    throw new ForbiddenException('Solo el creador del proyecto puede resolver postulaciones');
  return this.prisma.postulacion.update({
    where: { idPostulacion: id },
    data: {
      estadoPostulacion: dto.estadoPostulacion,
      comentarioResolucion: dto.comentarioResolucion ?? null,
      resueltaPor: resolutorId,
      fechaResolucion: new Date(),
    },
  });
}
```

T-37 — Vista de gestión de postulaciones del líder

Se desarrolló la página /dashboard/proyectos/[id]/postulaciones que permite al creador del proyecto visualizar todas las postulaciones recibidas agrupadas por rol. Cada fila muestra el nombre completo del postulante, su correo institucional, la justificación enviada, el estado actual de la postulación y, para las postulaciones pendientes, botones de acción para aceptar o rechazar. Al confirmar una acción, se dispara la mutación PATCH correspondiente y la vista se revalida automáticamente mediante TanStack Query.

5. HU-21: Notificación de resultado de postulación

T-38 — Implementación real del módulo de notificaciones

Se reemplazó el stub inicial del NotificationsService con una implementación completa. El servicio expone los métodos notifyUsers y notifyAdmins que persisten registros de notificación en la base de datos asociados a los usuarios destinatarios. Se implementó también el endpoint GET /notificaciones que devuelve las notificaciones del usuario autenticado ordenadas por fecha descendente, el endpoint GET /notificaciones/no-leidas que filtra solo las no leídas, y el endpoint PATCH /notificaciones/:id/leer que marca una notificación como leída registrando la fecha. El método markAllAsRead permite marcar todas las notificaciones pendientes de un usuario en una sola operación.

```
async notifyUsers(userIds: number[], payload, tx?) {
  if (userIds.length === 0) return;
  const db = tx ?? this.prisma;
  await db.notificacion.createMany({
    data: userIds.map(idUsuario => ({
      idUsuario,
      tipoNotificacion: payload.tipoNotificacion,
      tituloNotificacion: payload.tituloNotificacion,
      mensajeNotificacion: payload.mensajeNotificacion,
      datosJson: payload.datosJson,
    })),
    skipDuplicates: false,
  });
}

async markAsRead(id: number, userId: number) {
  // Valida que la notificación pertenezca al usuario
  if (notificacion.idUsuario !== userId)
    throw new ForbiddenException('No tienes permiso para esta notificación');
  return this.prisma.notificacion.update({
    where: { idNotificacion: id },
    data: { leidaEn: new Date() },
  });
}
```

T-39 — Generación automática de notificación al gestionar postulación

Se inyectó NotificationsService en el módulo de aplicaciones para que cada vez que el líder acepte o rechace una postulación se genere automáticamente una notificación dirigida al postulante. El mensaje varía según el resultado: si la postulación fue aceptada se incluye un mensaje de felicitación con el nombre del rol y del proyecto; si fue rechazada, se añade el comentario de resolución ingresado por el líder cuando este estuvo presente.

```
await this.notificationsService.notifyUsers(
  [postulacion.idUsuarioPostulante],
  {
    tipoNotificacion: TipoNotificacion.POSTULACION_RESUELTA,
    tituloNotificacion: esAceptada
      ? 'Tu postulación fue aceptada'
      : 'Tu postulación fue rechazada',
    mensajeNotificacion: esAceptada
      ? `Felicidades, tu postulación para el rol "${nombreRol}" en el proyecto "${tituloProyecto}" fue aceptada.`
      : `Tu postulación para el rol "${nombreRol}" en "${tituloProyecto}" fue rechazada.${
          dto.comentarioResolucion ? ` Comentario: ${dto.comentarioResolucion}` : ''}`,
    datosJson: { idPostulacion, idProyecto, idRolProyecto, estadoPostulacion: estado },
  },
);
```

T-40 — Componente de notificaciones en el dashboard

Se desarrolló el componente notifications-bell.tsx que se integra en la barra de navegación superior del DashboardLayout. El componente consulta periódicamente el endpoint GET /notificaciones/no-leidas y muestra un badge con el conteo de notificaciones pendientes. Al hacer clic se despliega un panel con el listado de notificaciones recientes, mostrando título, mensaje y fecha de cada una. Desde el panel el usuario puede marcar notificaciones individuales como leídas o utilizar la opción de marcar todas, lo que actualiza el badge en tiempo real.

6. HU-17: Búsqueda avanzada de proyectos

T-41 — Extensión del endpoint GET /proyectos con filtros adicionales

Se extendió el endpoint GET /proyectos para aceptar cuatro parámetros de consulta adicionales: tipoProyecto, modalidad, organizacionId y habilidad. El servicio construye dinámicamente un arreglo de condiciones AND que se compone según los filtros presentes, manteniendo compatibilidad con el filtro por título (q) implementado en el Sprint 1. El filtro por habilidad realiza una búsqueda profunda en la estructura roles → requisitos → habilidad, lo que permite a los estudiantes encontrar proyectos que requieran competencias específicas.

```
// Filtros dinámicos en findAll()
const andConditions: Prisma.ProyectoWhereInput[] = [
  { estadoProyecto: { in: ESTADOS_VISIBLES } },
  { eliminadoEn: null },
];
```

```
if (q) andConditions.push({
  tituloProyecto: { contains: q.trim(), mode: 'insensitive' }
});
if (tipoProyecto) andConditions.push({ tipoProyecto: tipoProyecto as TipoProyecto });
if (modalidad) andConditions.push({ modalidadProyecto: modalidad as ModalidadProyecto
});
if (organizacionId) andConditions.push({
  organizaciones: { some: { idOrganizacion: organizacionId } }
});
if (habilidadId) andConditions.push({
  roles: { some: { requisitos: { some: { idHabilidad: habilidadId } } } }
});
return this.prisma.proyecto.findMany({
  where: { AND: andConditions },
  select: proyectoListSelect,
  orderBy: { fechaCreacion: 'desc' }, take: 20,
});
```

T-42 — Componente de filtros en la vista de listado de proyectos

Se agregó un panel de filtros colapsable en la vista /dashboard/proyectos que complementa el campo de búsqueda por título existente. El panel expone selectores para tipo de proyecto, modalidad, organización y habilidad. Al cambiar cualquier filtro, los parámetros se propagan al hook useQuery que consulta el endpoint GET /proyectos, actualizando la lista de resultados en tiempo real sin necesidad de recargar la página. Los filtros activos se reflejan visualmente con badges para que el usuario identifique fácilmente los criterios aplicados.

7. HU-25: Cambiar estado del proyecto (Vista Mis Proyectos)

T-43 — Endpoint GET /proyectos/mis-proyectos

Se implementó el endpoint protegido GET /proyectos/mine (con alias GET /proyectos/mis-proyectos para compatibilidad) que devuelve todos los proyectos creados por el usuario autenticado, independientemente de su estado. La respuesta incluye métricas calculadas: cantidadPostulaciones (suma de postulaciones recibidas en todos los roles), rolesCubiertos (cantidad de roles con al menos una participación activa) y rolesTotales. También incluye la última revisión del proyecto para que el líder conozca el estado del proceso de aprobación.

```
async findMine(userId: number) {
  const proyectos = await this.prisma.proyecto.findMany({
    where: { creadoPor: userId, eliminadoEn: null },
    select: {
      ...proyectoListSelect,
```

```
      fechaCreacion: true, fechaActualizacion: true,
      roles: {
        select: { idRolProyecto: true, _count: {
          select: { postulaciones: true, participaciones: true }
        }}
      },
      revisiones: { select: { estadoRevision: true, comentarioRevision: true,
                              numeroEnvio: true, enviadaEn: true, revisadaEn: true },
                    orderBy: { enviadaEn: 'desc' }, take: 1 },
    },
    orderBy: { fechaCreacion: 'desc' },
  });
  return proyectos.map(p => ({
    ...p,
    cantidadPostulaciones: p.roles.reduce((a, r) => a + r._count.postulaciones, 0),
    rolesCubiertos: p.roles.filter(r => r._count.participaciones > 0).length,
    rolesTotales: p.roles.length,
  }));
}
```

T-44 — Vista Mis Proyectos con control de estado

Se desarrolló la página /dashboard/mis-proyectos que lista todos los proyectos creados por el usuario autenticado. Cada tarjeta muestra el título, estado con badge de color, tipo, modalidad, fecha de creación o publicación, cantidad de postulaciones recibidas y la proporción de roles cubiertos. Desde la misma tarjeta el líder puede cambiar el estado del proyecto mediante un selector que despacha el endpoint PATCH /proyectos/:id/estado. Solo se presentan las transiciones válidas. La tarjeta también incluye un enlace directo a la vista de gestión de postulaciones recibidas del proyecto.

```
// Selector de estado en la tarjeta de proyecto
<Select
  value={selectedStatus}
  onValueChange={(value) => onSelectStatus(value as ProjectStatus)}
  disabled={isUpdating}
>
  <SelectTrigger className="h-9 w-[160px] text-sm">
    <Settings2 className="mr-1.5 h-3.5 w-3.5 text-muted-foreground" />
    <SelectValue />
  </SelectTrigger>
  <SelectContent>
    {statusOptions.map(status => (
      <SelectItem key={status} value={status}>{statusConfig[status].label}</SelectItem>
    ))}
  </SelectContent>
</Select>
```

8. HT-02: Tareas transversales del Sprint 2

T-45 — Configuración de ESLint en el backend

Se añadió ESLint con reglas TypeScript strict en el backend de NestJS. El frontend ya contaba con su propio archivo de configuración eslint.config.mjs desde el Sprint 1. Se corrigieron los errores de linting existentes en el código del backend y se incorporó el script lint al package.json del proyecto para que pueda ejecutarse de forma independiente como parte de los flujos de integración continua.

T-46 — Mejora de consistencia visual de la UI

Se estandarizaron los tamaños de formularios, listados y tarjetas en todas las vistas del sprint según el feedback recibido durante las pruebas internas. Se ajustó el espaciado entre componentes, la tipografía de etiquetas y la alineación de elementos en las vistas de perfil, creación de proyectos, listado de proyectos y gestión de postulaciones para lograr una experiencia visual coherente en toda la aplicación.

T-47 — Pruebas del flujo completo del Sprint 2

Las pruebas del flujo completo del Sprint 2 se realizaron de forma manual recorriendo los siguientes escenarios de extremo a extremo: completar y editar el perfil académico (incluyendo habilidades, intereses y cualidades), crear un proyecto con roles y enviarlo a revisión, postularse a un rol de un proyecto publicado, gestionar postulaciones recibidas como líder aceptando y rechazando, verificar la recepción de notificaciones tras cada resolución, buscar proyectos aplicando los nuevos filtros combinados, y visualizar y cambiar el estado de un proyecto desde la vista Mis Proyectos.

T-48 — Corrección de errores generales

Durante la integración y pruebas del sprint se identificaron y corrigieron errores en los flujos de los módulos desarrollados. Las correcciones abarcaron ajustes de responsividad en el formulario de creación de proyecto, mejoras en el manejo de errores de red en las mutaciones de TanStack Query para que los mensajes de error sean descriptivos al usuario, y la resolución de inconsistencias en los estados iniciales de los formularios al cambiar entre modo creación y modo edición.

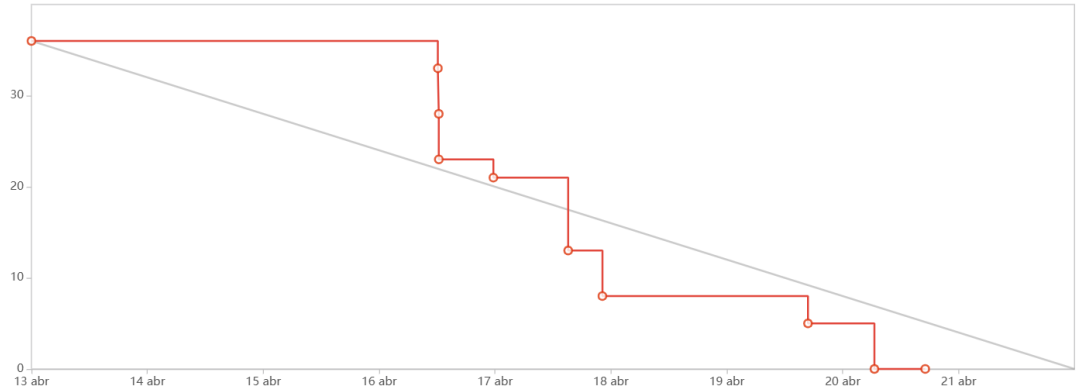
T-49 — Preparación de presentación y guion

La preparación de la presentación del Sprint 2 se realizó utilizando herramientas externas de colaboración del equipo. Se elaboraron las diapositivas con el sistema desarrollado, las decisiones técnicas tomadas durante el sprint y los resultados del burndown. El guion fue definido de manera colaborativa para determinar el orden de exposición y la demostración en vivo de los flujos implementados. Esta actividad no genera artefactos directamente en el repositorio de código.

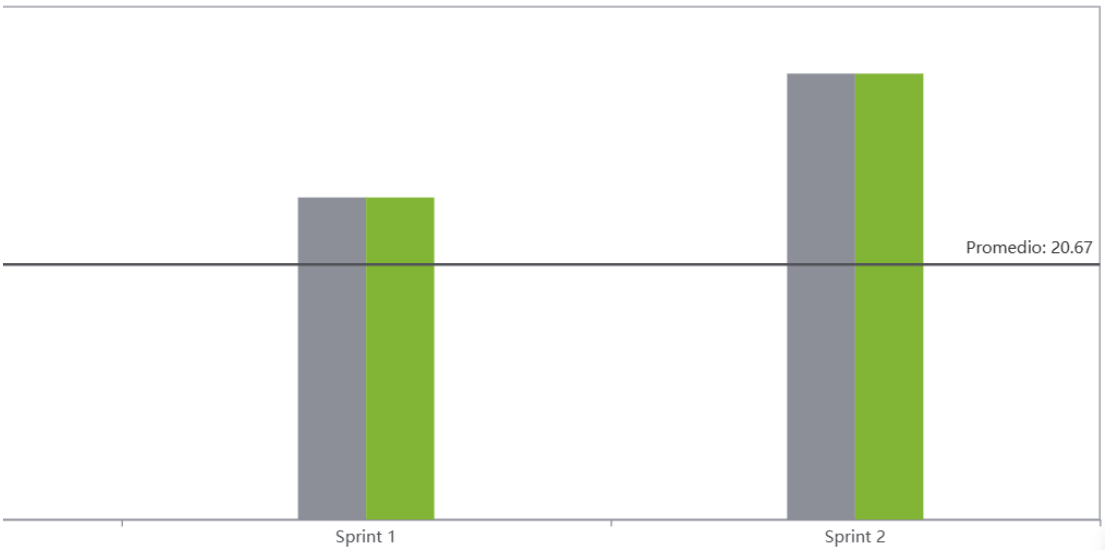
Las funcionalidades descritas anteriormente se encuentran integradas en la rama principal del repositorio y pueden ser ejecutadas mediante los archivos de Docker descritos en la documentación de infraestructura del Sprint 1.

Resultados del Sprint

Gráfica Burndown



Métrica de velocidad



Discusión

El sprint correspondiente al período del 13 al 22 de abril de 2026 puede considerarse exitoso, ya que el equipo logró completar el 100% de los 35 puntos comprometidos y cumplió con la meta de entregar una versión estable del MVP incluyendo notificaciones, gestión de proyectos y el nuevo flujo de postulaciones. La velocidad del equipo mostró una tendencia positiva al pasar de 26 puntos en el Sprint 1

a 36 en el Sprint 2, superando el promedio histórico de 20.67 puntos. Sin embargo, el burndown chart revela un patrón de back-loading, ya que los primeros tres días no registraron avances y la mayor parte del trabajo se completó en el tramo final del sprint, lo cual representa un riesgo que debe atenderse en futuras iteraciones mediante una mejor división de historias y una distribución más uniforme de las entregas a lo largo de los días del sprint.

Retrospectiva del Sprint

Aspectos positivos

- **Cumplimiento total del alcance por segundo sprint consecutivo:** Se completaron los 35 puntos de historia comprometidos (100%), lo que consolida la capacidad del equipo para estimar y cumplir con el alcance planificado. Este resultado es especialmente significativo considerando que la carga aumentó un 34.6% respecto al Sprint 1 (de 26 a 35 puntos).
- **Incremento notable de la velocidad del equipo:** La velocidad pasó de 26 puntos en el Sprint 1 a 35 puntos en el Sprint 2, superando el promedio histórico de 20.67 puntos. Esto evidencia que el equipo ganó confianza y fluidez con el stack tecnológico y con las dinámicas de trabajo establecidas en la iteración anterior.
- **Mayor complejidad funcional abordada con éxito:** Mientras que el Sprint 1 cubrió un flujo lineal básico (autenticación → exploración → postulación), el Sprint 2 incorporó funcionalidades transversales más complejas como la creación de proyectos con roles y requisitos, y la búsqueda avanzada con filtros dinámicos. La entrega de estas funcionalidades demuestra madurez en el manejo del stack y en la coordinación del equipo.
- **Mejora en la calidad del código:** Se cumplió con la medida correctiva del Sprint 1 de incorporar ESLint con reglas estrictas de TypeScript en el backend, lo que estandariza la calidad del código y facilita la detección temprana de errores. El frontend ya contaba con esta configuración desde el sprint anterior.
- **Consistencia visual mejorada:** Se realizó un esfuerzo deliberado de estandarización de la interfaz (T-46), atendiendo el feedback recibido durante pruebas internas y ajustando formularios, listados y tarjetas para lograr una experiencia coherente en todas las vistas del sistema.
- **Continuidad en la división clara de responsabilidades:** Se mantuvo la práctica de asignar tareas diferenciadas a cada integrante, evitando que más de dos personas trabajaran en la misma funcionalidad y permitiendo avance en paralelo sin conflictos de integración significativos.
- **Se formalizó el proceso de code review:** Se estableció como requisito que todo pull request hacia develop cuente con al menos una aprobación de otro integrante del equipo antes del merge, documentando esta práctica en el flujo de trabajo del repositorio.

Aspectos negativos y oportunidades de mejora

- **Persistencia del patrón de back-loading en el burndown:** A pesar de que en la retrospectiva del Sprint 1 se identificó la concentración del trabajo en la segunda mitad del sprint como un riesgo y se estableció como medida correctiva el cierre incremental de tareas desde el primer día, el

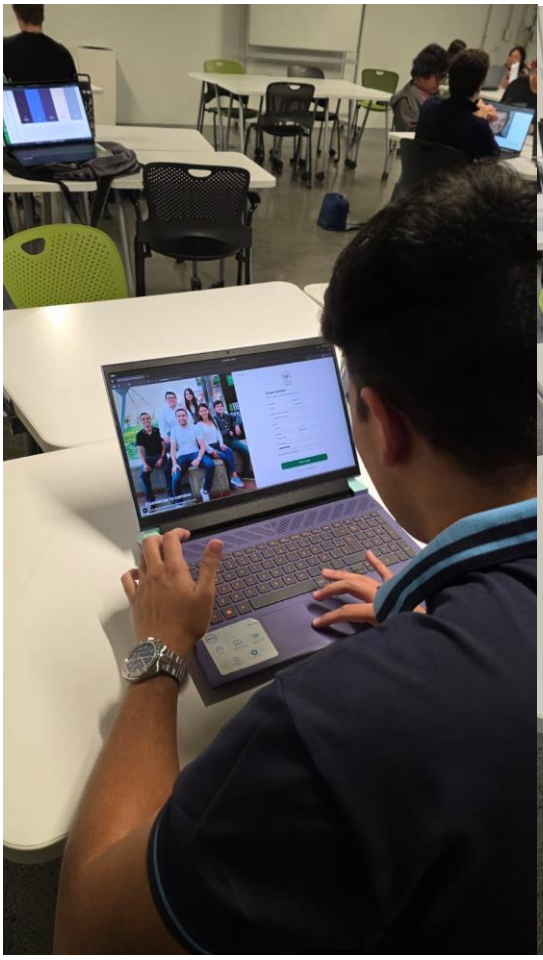
burndown del Sprint 2 muestra el mismo patrón: los primeros tres días (13 al 15 de abril) no registraron avances, y la totalidad del trabajo se completó entre el 16 y el 21 de abril. Esto indica que la medida correctiva no se implementó de manera efectiva y que el equipo continúa posponiendo el cierre formal de tareas, lo cual incrementa el riesgo de acumulación de errores en los últimos días y dificulta la detección temprana de impedimentos.

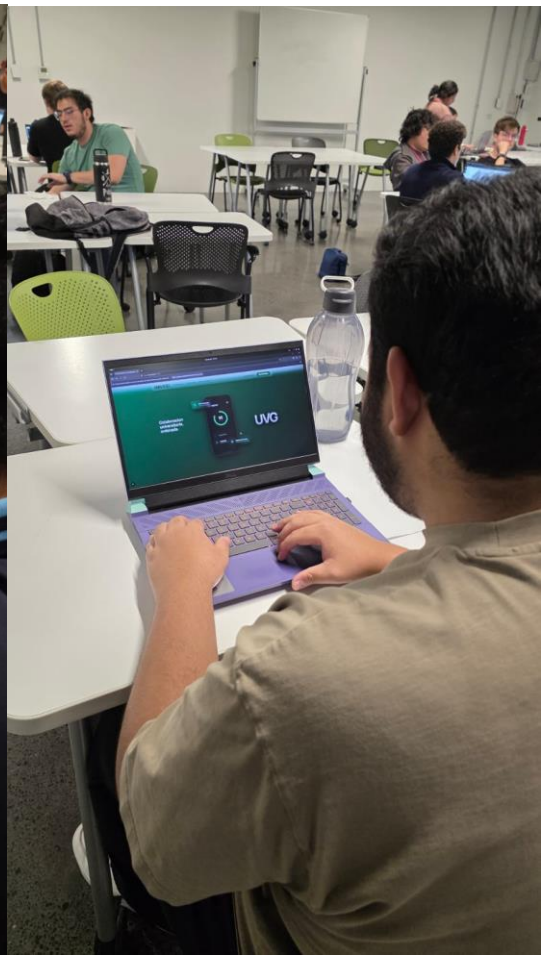
- **Ausencia de pruebas automatizadas por segundo sprint consecutivo:** En el Sprint 1 se estableció como medida correctiva la incorporación de pruebas automatizadas básicas para los endpoints principales del backend. Sin embargo, las pruebas del Sprint 2 (T-47) se realizaron nuevamente de forma manual. Esta deuda técnica se acumula conforme el sistema crece en funcionalidades y aumenta el riesgo de regresiones no detectadas en sprints futuros.
- **Falta de evidencia sobre revisiones de código formales:** En la retrospectiva del Sprint 1 se propuso realizar al menos una sesión de code review formal por cada pull request antes de mergearlo a develop. Práctica nueva que no se implementó de forma sistemática.
- **Manejo de errores reactivo en lugar de preventivo:** Las correcciones realizadas en T-48 (ajustes de responsividad, mejoras en el manejo de errores de red y resolución de inconsistencias en estados iniciales de formularios) evidencian que algunos defectos se detectaron durante la integración final y no durante el desarrollo individual de cada tarea, lo cual es consecuencia directa de la ausencia de pruebas automatizadas y del patrón de integración tardía.

Medidas a tomar para el Sprint 3

1. **Establecer entregables intermedios obligatorios:** Definir al menos dos checkpoints durante el sprint en los que cada integrante debe tener al menos una tarea en estado Done en Jira, para forzar el cierre incremental y evitar la acumulación al final.
2. **Incorporar pruebas automatizadas como criterio de aceptación:** Implementar al menos pruebas de integración para los endpoints principales del backend utilizando el módulo de testing de NestJS. Esta medida no debe posponerse por tercer sprint consecutivo.
3. **Dividir historias de usuario en tareas más granulares:** Reducir el tamaño de las tareas individuales para facilitar su cierre temprano y que el burndown refleje un progreso más uniforme a lo largo del sprint.
4. **Registrar y documentar las pruebas realizadas:** Crear un documento o checklist de pruebas manuales ejecutadas durante el sprint como evidencia del proceso de validación, mientras se avanza en la automatización.

Evidencias de muestra del incremento a usuarios





El incremento desarrollado durante el Sprint 2 fue presentado a estudiantes de diversas asociaciones estudiantiles de la universidad, incluyendo tanto miembros activos como estudiantes interesados en participar en proyectos colaborativos. Se les mostró el flujo completo de las nuevas funcionalidades: creación de proyectos y los filtros avanzados de búsqueda. Los comentarios recibidos destacaron la claridad del formulario de creación de proyectos en tres pasos, mientras que se sugirieron mejoras en la visibilidad de los filtros activos en la vista de búsqueda y en la retroalimentación visual al cambiar el estado de un proyecto.

Indicador Numérico del Éxito del Sprint 2

1. Definición del Indicador

El Índice de Éxito del Sprint (IES) es una métrica compuesta diseñada para evaluar de manera integral el desempeño del equipo en cada iteración del proyecto. A diferencia de métricas simples como el porcentaje de cumplimiento, el IES incorpora múltiples dimensiones que reflejan no solo si el equipo entregó lo prometido, sino también cómo lo entregó, con qué calidad, y si el resultado fue validado por los usuarios finales.

El indicador se expresa en una escala de 1 a 10, donde 1 representa un fracaso total y 10 un éxito rotundo. Su diseño permite la comparación objetiva entre sprints sucesivos y la identificación precisa de las áreas que requieren atención correctiva.

2. Fórmula del IES

$$\mathbf{IES = (C \times 0.30) + (V \times 0.15) + (Q \times 0.15) + (D \times 0.10) + (P \times 0.15) + (U \times 0.15)}$$

Donde cada variable representa una dimensión evaluada en escala de 1 a 10 y multiplicada por su peso correspondiente. La suma de todos los pesos es igual a 1.00 (100%), lo que garantiza que el resultado final permanezca en la escala de 1 a 10.

3. Dimensiones, siglas y pesos

Cada dimensión del IES fue seleccionada para cubrir un aspecto distinto del desempeño del sprint. Las siglas corresponden al nombre de cada dimensión en español (o en inglés para Quality), y los pesos fueron asignados según el impacto relativo de cada factor en el éxito real de una iteración ágil. A continuación se explica cada una.

- **3.1 C — Cumplimiento del alcance (peso: 30%)**

  Sigla: C, derivada de «Cumplimiento».

  Qué mide: la proporción de puntos de historia completados respecto a los comprometidos al inicio del sprint. Se calcula objetivamente como (SP completados / SP comprometidos) × 10.

  Por qué tiene el mayor peso (30%): en Scrum, el objetivo principal de cada sprint es entregar el incremento comprometido. Si el equipo no completa los story points planificados, el resto de las dimensiones pierde relevancia porque no existe un producto funcional que evaluar. El cumplimiento es el pilar fundamental sobre el que se construyen todas las demás métricas. Se le asignó casi un tercio del peso total porque sin cumplimiento no hay incremento entregable y, por tanto, no hay éxito.

- **3.2 V — Tendencia de velocidad (peso: 15%)**

  Sigla: V, derivada de «Velocidad».

Qué mide: si la capacidad productiva del equipo está mejorando, estancada o deteriorándose con respecto a sprints anteriores. Compara la velocidad actual (SP completados) contra el promedio histórico y la tendencia de los sprints previos.

Por qué pesa 15%: un equipo que cumple el alcance de un sprint aislado pero cuya velocidad decrece consistentemente está perdiendo capacidad. Sin embargo, la velocidad no define por sí sola el éxito de una iteración individual: un equipo podría reducir deliberadamente su velocidad para abordar deuda técnica sin que ello constituya un fracaso. Por esta razón tiene un peso moderado, suficiente para detectar tendencias pero sin castigar decisiones estratégicas legítimas.

- **3.3 Q — Calidad del incremento (peso: 15%)**

  Sigla: Q, derivada de «Quality» (calidad en inglés).

  Qué mide: la severidad y cantidad de defectos encontrados durante las pruebas de integración y validación del sprint. Evalúa si el código entregado es realmente funcional y estable o si presenta errores que comprometen la experiencia del usuario.

  Por qué pesa 15%: calidad y velocidad son fuerzas complementarias. Un sprint puede completar el 100% de los story points, pero si el código resultante está plagado de defectos severos, el incremento no es verdaderamente entregable. Se le asignó el mismo peso que a la velocidad porque ambas dimensiones se equilibran mutuamente: si calidad pesara más que velocidad se penalizaría a equipos que avanzan rápido; si pesara menos, se premiaría la entrega de código defectuoso.

- **3.4 D — Distribución del trabajo (peso: 10%)**

  Sigla: D, derivada de «Distribución».

  Qué mide: la forma del gráfico burndown, es decir, si el cierre de tareas se distribuyó de manera uniforme a lo largo del sprint o si todo el trabajo se concentró en los últimos días (patrón de back-loading).

  Por qué tiene el peso más bajo (10%): la distribución del trabajo es un indicador de proceso, no de resultado. Un equipo que presenta back-loading pero aun así entrega el 100% del alcance a tiempo no fracasó, aunque sí está asumiendo un riesgo innecesario al comprimir el tiempo de pruebas y correcciones. Si esta dimensión tuviera un peso mayor, castigaría desproporcionadamente a equipos que entregan todo lo comprometido solo porque su ritmo de cierre no fue lineal. No obstante, mantiene un 10% porque el patrón de back-loading es una señal de alerta legítima que debe ser visibilizada.

- **3.5 P — Prácticas de ingeniería (peso: 15%)**

  Sigla: P, derivada de «Prácticas».

  Qué mide: la adopción y ejecución de prácticas que aseguran la sostenibilidad del código a largo plazo: linting con reglas estrictas, pruebas automatizadas, revisiones de código formales (code review) e integración continua.

Por qué pesa 15%: es lo que diferencia un proyecto que puede escalar de uno que se desmoronará en pocos sprints más. En un contexto académico con cuatro sprints totales la deuda técnica no alcanza a explotar con la misma gravedad que en producción real, por lo que no se le asignó un peso mayor. Sin embargo, tiene suficiente presencia como para que su ausencia parcial o total baje visiblemente el indicador, incentivando al equipo a incorporar estas prácticas progresivamente.

- **3.6 U — Retroalimentación de usuarios (peso: 15%)**

  Sigla: U, derivada de «Usuarios».

  Qué mide: si el incremento entregado fue validado positivamente por usuarios reales o representativos. En Scrum el valor lo define el usuario, no el desarrollador; por tanto, un sprint que entrega funcionalidades que los usuarios no consideran útiles no puede considerarse plenamente exitoso.

  Por qué pesa 15%: valida que el equipo no solo construyó lo planificado, sino que construyó algo que realmente resuelve una necesidad. No se le asignó un peso mayor porque, en el contexto del proyecto, la socialización se realizó con una muestra limitada de estudiantes y no con un product owner formal con autoridad decisoria sobre el producto. Un peso del 15% es suficiente para que la validación de usuarios influya en el indicador sin sobredimensionar una retroalimentación que, por su naturaleza muestral, no es estadísticamente representativa.

4. **Distribución por categorías**

Los pesos de las seis dimensiones pueden agruparse en tres categorías conceptuales que reflejan las prioridades del marco Scrum:

Categoría	Dimensiones	Peso combinado
Resultado directo	C + V	45%
Calidad y sostenibilidad	Q + P	30%
Proceso y validación	D + U	25%

La distribución 45-30-25 refleja la filosofía de Scrum: primero entregar valor funcional al usuario, luego asegurar que lo entregado sea de calidad y mantenible, y finalmente evaluar la eficiencia del proceso de trabajo y la validación externa. Esta jerarquía garantiza que el indicador nunca premie un proceso impecable si no hubo entrega, ni castigue una entrega completa solo por deficiencias menores en el proceso.

5. **Aplicación al Sprint 2**

A continuación se presenta la evaluación del Sprint 2 (13 al 22 de abril de 2026) utilizando la fórmula del IES:

Dimensión	Sigla	Peso	Calificación	Ponderado	Justificación clave
Cumplimiento del alcance	C	30%	10	3.00	35/35 SP (100%)
Tendencia de velocidad	V	15%	10	1.50	26 → 35 SP (+34.6%)
Calidad del incremento	Q	15%	9	1.35	Defectos leves, reactivos
Distribución del trabajo	D	10%	6	0.60	Back-loading persistente
Prácticas de ingeniería	P	15%	8	1.20	ESLint sí, testing no
Retroalimentación de usuarios	U	15%	9	1.35	Feedback positivo
TOTAL		**100%**		**9.00**	

6. **Justificación detallada por dimensión**

Cumplimiento del alcance: 10/10

Se completaron los 35 puntos de historia comprometidos, alcanzando un 100% de cumplimiento por segundo sprint consecutivo. No quedaron historias de usuario en proceso ni inconclusas. Las ocho historias del sprint backlog (HU-02, HU-13, HU-07, HU-08, HU-21, HU-17, HU-25 y HT-02) fueron entregadas con su funcionalidad completa integrada en la rama principal del repositorio.

Tendencia de velocidad: 10/10

La velocidad del equipo pasó de 26 puntos de historia en el Sprint 1 a 35 puntos en el Sprint 2, lo que representa un incremento del 34.6%. Este resultado supera ampliamente el promedio histórico de 20.67 puntos y confirma una tendencia ascendente en la capacidad productiva del equipo, atribuible a la mayor familiaridad con el stack tecnológico y a la consolidación de las dinámicas de trabajo.

Calidad del incremento: 9/10

Los defectos identificados durante la integración (documentados en la tarea T-48) fueron de baja severidad: ajustes de responsividad en el formulario de creación de proyecto, mensajes de error de red insuficientemente descriptivos e inconsistencias en los estados iniciales de formularios al alternar entre modo creación y edición. Todos fueron corregidos antes del cierre del sprint. Se descuenta un punto porque la detección fue reactiva (durante la integración final) y no preventiva, lo que evidencia la ausencia de pruebas automatizadas.

Distribución del trabajo: 6/10

El gráfico burndown muestra nuevamente un patrón de back-loading: los primeros tres días del sprint (13 al 15 de abril) no registraron avances formales y la totalidad del trabajo se cerró entre el día 4 y el día 9. Esta es la dimensión con peor desempeño y la principal razón por la cual el indicador no alcanza el 10. El patrón ya había sido identificado en la retrospectiva del Sprint 1 y la medida correctiva propuesta (cierre incremental desde el primer día) no se implementó de manera efectiva.

Prácticas de ingeniería: 8/10

Se cumplió con la incorporación de ESLint con reglas TypeScript strict en el backend, atendiendo la medida correctiva del Sprint 1. Sin embargo, las pruebas del flujo completo (T-47) se realizaron nuevamente de forma manual, sin incorporar pruebas automatizadas por segundo sprint consecutivo. El proceso de code review se formalizó como requisito para pull requests pero no se evidenció su ejecución sistemática. La ausencia de testing automatizado y la falta de evidencia de code review impiden alcanzar una calificación superior.

Retroalimentación de usuarios: 9/10

El incremento fue socializado con estudiantes de diversas asociaciones estudiantiles de la universidad. Los comentarios recibidos fueron predominantemente positivos, destacando la claridad del formulario de creación de proyectos en tres pasos. Las sugerencias recibidas (mejorar visibilidad de filtros activos y retroalimentación visual al cambiar estado de proyecto) son mejoras de usabilidad menores que no comprometen la funcionalidad del sistema. Se descuenta un punto por el alcance limitado de la muestra de usuarios consultados.

7. Escala de interpretación

Para estandarizar la interpretación del IES a lo largo del proyecto, se establece la siguiente escala:

Rango	Clasificación	Acción requerida
8.0 – 10.0	Exitoso	Mantener prácticas y atender mejoras menores identificadas.
6.0 – 7.9	Aceptable con reservas	Implementar medidas correctivas específicas antes del siguiente sprint.

1.0 – 5.9	Requiere intervención	Detener y reevaluar planificación, alcance y prácticas de trabajo.

Con un IES de 9.00, el Sprint 2 se clasifica como exitoso. La calificación refleja que el equipo entregó la totalidad del alcance comprometido con calidad aceptable y validación positiva de usuarios, pero con oportunidades de mejora claras en la distribución del trabajo y en las prácticas de ingeniería.

8. Replicabilidad para sprints futuros

La fórmula del IES está diseñada para ser aplicada de manera idéntica en cada sprint del proyecto, lo que permite comparar la evolución del indicador a lo largo del tiempo. Los pesos pueden ajustarse si el equipo decide priorizar una dimensión específica en un sprint determinado. Por ejemplo, si en el Sprint 3 la meta principal es resolver el back-loading, el equipo podría incrementar el peso de D de 10% a 15% y reducir V de 15% a 10%, manteniendo la suma total en 100%.

Cualquier ajuste de pesos debe documentarse y justificarse antes del inicio del sprint para evitar manipulaciones retroactivas del indicador. La calificación de cada dimensión debe basarse en evidencia objetiva (métricas del burndown, conteo de defectos, resultados de socialización) y no en apreciaciones subjetivas del equipo.

9. Resultado final

El indicador numérico de éxito del sprint (IES = 9.0/10) confirma cuantitativamente que el Sprint 2 fue exitoso, pero también expone con precisión dónde se concentra la deuda: la distribución del trabajo (6/10) y las prácticas de ingeniería (8/10) son las dos dimensiones que impiden alcanzar la calificación máxima. El valor del indicador no radica únicamente en el número final, sino en que descompone el desempeño en factores accionables que orientan las medidas correctivas del Sprint 3. Si el equipo logra mejorar la distribución del burndown a un 8/10 e incorporar pruebas automatizadas para elevar las prácticas de ingeniería a un 9/10, el IES podría alcanzar 9.5 o superior en la próxima iteración.

La arquitectura técnica del sistema demostró ser lo suficientemente robusta como para soportar el crecimiento funcional del Sprint 2 sin requerir refactorizaciones estructurales. El uso de transacciones Prisma para operaciones compuestas, la construcción dinámica de filtros con condiciones AND y la inyección del servicio de notificaciones en el módulo de postulaciones evidencian que las decisiones de diseño tomadas en el Sprint 1 escalaron correctamente ante una carga funcional significativamente mayor.

La persistencia del patrón de back-loading por segundo sprint consecutivo sugiere que el problema no es de planificación sino de hábitos de cierre. El equipo demuestra capacidad para completar el alcance comprometido, pero concentra la integración y el cierre formal de tareas en los últimos días, lo cual comprime el tiempo disponible para pruebas y correcciones. Las medidas correctivas propuestas para el Sprint 3 atacan directamente este patrón al forzar cierres parciales tempranos que distribuyan el riesgo a lo largo del sprint en lugar de acumularlo al final.

Conclusiones

- En resumen, puede afirmarse que el proyecto continúa avanzando de manera sólida a partir de los resultados obtenidos en el Sprint 2. Este progreso no se explica únicamente por la cantidad de funcionalidades incorporadas, sino también por una comprensión más clara del problema que se busca resolver, por la construcción de una solución cada vez más útil para los usuarios y por una mayor madurez en el proceso de desarrollo. A partir de esta iteración, el principal desafío no será solamente agregar nuevas funcionalidades, sino hacerlo de manera más equilibrada, con validaciones más formales y con una estrategia de trabajo que favorezca la sostenibilidad del proyecto conforme aumente su complejidad.

- El Sprint 2 representó un avance significativo en el desarrollo del proyecto, ya que permitió la consolidación de funciones esenciales para fortalecer la plataforma y optimizar la experiencia de los usuarios. La aplicación de módulos como la gestión de postulaciones, las alertas, el manejo del perfil académico, así como la creación y administración de proyectos, evidencia un incremento importante en el MVP, tanto en lo que respecta a su grado de integración entre sus componentes como a su cobertura funcional. Además, que se cumplan todos los acuerdos demuestra un progreso positivo en la capacidad del equipo para coordinarse y ofrecer resultados concretos al final de la iteración.

- Al mismo tiempo, los resultados del sprint indican que el método de trabajo debe cambiar para poder mantener el ritmo de crecimiento del proyecto. La experiencia adquirida en esta iteración evidenció que no basta con lograr el alcance previsto; es necesario, además, formalizar prácticas y validar de manera constante para afianzar la distribución del trabajo y asegurar la calidad del producto a lo largo del tiempo. Por ende, el valor del Sprint 2 no solo se basa en las funcionalidades que se han implementado, sino también en los conocimientos obtenidos para dirigir de mejor manera las decisiones y prioridades de los ciclos posteriores de desarrollo.